MACHINE LEARNING PROJECT

Machine Learning

Credit risk classification: from preprocessing to SAFE AI metrics

Federico Crivellari | Alan Sampaolo

Objective and workflow

OVERVIEW

Objective. Build and compare classification models that could be used to predict the credit risk of future loan applicants, identifying the model that best separates higher-risk from lower-risk loans on unseen data.

01
Preprocessing
Feature engineering, encoding and scaling

02
Unsupervised
Clustering and PCA visualisation

03
Supervised
Leakage-free pipeline and model comparison

04
SAFE AI
RGE explainability and RGR robustness

Feature engineering

PREPROCESSING

Transform raw variables

New predictors were created to capture leverage, credit history and borrower quality.

Examples
Credit age
Credit usage ratio
Loan-to-income ratio
Average FICO

```
df["credit_age"] = 2024 - df["earliest_cr_line"].dt.year
df["credit_usage_ratio"] = (
    df["revol_bal"] / (df["total_rev_hi_lim"] + 1))
df["loan_to_income_ratio"] = (
    df["loan_amnt"] / (df["annual_inc"] + 1))
df["fico_avg"] = (
    df["fico_range_low"] + df["fico_range_high"]) / 2
```

Selected features

PREPROCESSING

01
Loan characteristics
Loan amount
Interest rate
Installment
Grade
DTI

02
Credit profile
FICO score
Credit age
Open accounts
Total accounts
Public records

03
Engineered features
Credit usage
Record indicator
Recent activity
Loan / income
Average FICO

Ordinal variables

ENCODING

Label encoding

Loan grade is ordinal, so its natural ranking should be retained.

$$A > B > C > D > E > F > G$$

The categories are mapped to a decreasing numerical scale.

```
df_filtered["grade"] = (
    df_filtered["grade"].str.strip().str.upper())
mapping_grade = {"A": 6, "B": 5, "C": 4, "D": 3,
                 "E": 2, "F": 1, "G": 0}
df_filtered["grade"] = (
    df_filtered["grade"].map(mapping_grade))
```

Nominal variables

ENCODING

One-hot encoding

Home ownership is nominal, therefore no ranking should be imposed.

Mortgage	Rent
Own	Any
Other	None

```
df_filtered = pd.get_dummies(
    df_filtered, columns=["home_ownership"],
    drop_first=False)
dummy_cols = [c for c in df_filtered.columns
              if c.startswith("home_ownership_")]
df_filtered[dummy_cols] = df_filtered[dummy_cols].astype(int)
```

Missing values

DATA QUALITY

Missing observations were inspected before the modelling stage and the dataset was cleaned accordingly.

Missing values detected

Variable	Missing
DTI	0.028%
inq_last_6mths	<0.001%
Credit usage ratio	5.02%
Public record score	0.052%
Recent activity score	5.02%

Largest concentration: approximately 5%.

Exploratory data analysis

EDA

Distribution analysis
- Histograms
- Boxplots
- Frequency plots
- Outlier detection

Dependence analysis
- Correlation matrix
- Heat map
- Feature relationships
- Target behaviour

EDA was used to understand distributions, outliers and relationships among predictors.

Standardisation or normalisation?

SCALING

Standardisation

$$z_i = \frac{x_i - \mu}{\sigma}$$

Centres the variable around zero and rescales it to unit standard deviation.

Normalisation

$$x_i^* = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

Maps the variable into a fixed interval, typically $[0, 1]$.

Our choice: Normalisation (min-max scaling)

Target variable

CLASS IMBALANCE



4:1

Fully paid loans are about four times more frequent than charged-off loans (default rate in the test set: 19.6%).

Class imbalance must therefore be considered both in training and in evaluation.

Clustering

UNSUPERVISED LEARNING

Cluster summary

```
cluster_summary = (
    X_sample.groupby("cluster")
            .mean(numeric_only=True))
print(cluster_summary)
```

Group averages help interpret the characteristics of each K-Means cluster.

PCA visualisation

```
pca = PCA(n_components=2)
X_sample_pca = pca.fit_transform(X_sample)
pca_df = pd.DataFrame(X_sample_pca,
                      columns=["PC1", "PC2"])
pca_df["cluster"] = km.labels_
```

PCA reduces the feature space to two dimensions for visualisation. Clusters are exploratory and not used as predictors.

A leakage-free pipeline

SUPERVISED LEARNING

Sample and split
Random sample of 50,000 loans, stratified 80/20 train–test split. The test set (10,000 loans) is used only once, at the end.

SMOTE inside the pipeline
Oversampling is applied only to the training folds. Validation folds and the test set keep the real class distribution, so there is no data leakage.

Stratified 5-fold cross-validation
Each fold preserves the default rate of the training data.

```
from imblearn.pipeline import Pipeline
cv = StratifiedKFold(n_splits=5, shuffle=True,
                     random_state=42)
pipe = Pipeline([("smote", SMOTE(random_state=42)),
                 ("model", model)])
clf = GridSearchCV(pipe, params,
                   scoring="roc_auc", cv=cv)
clf.fit(X_train, y_train)      # SMOTE on train only
clf.predict_proba(X_test)     # SMOTE skipped
```

Evaluation metrics

SUPERVISED LEARNING

ROC-AUC
Probability that a defaulted loan is ranked as riskier than a repaid one.
Random model: 0.5

PR-AUC
Precision–recall trade-off on the default class: more informative with imbalanced data.
Random model: 0.196

Precision / Recall
Share of flagged loans that default / share of defaults that are flagged.
At threshold 0.5

Accuracy alone is misleading: a model that always predicts "no default" would reach about 80% accuracy while detecting no defaults.

Regularisation

SUPERVISED LEARNING

Ridge — L2

$$\mathcal{L}_{\text{Ridge}} = \mathcal{L}_0 + \lambda \sum_{j=1}^{p} \beta_j^2$$

Shrinks large coefficients towards zero (used in our logistic regression).

Lasso — L1

$$\mathcal{L}_{\text{Lasso}} = \mathcal{L}_0 + \lambda \sum_{j=1}^{p} |\beta_j|$$

Can reduce some coefficients exactly to zero.

Logistic Regression

MODEL RESULTS (TEST SET, 10,000 LOANS)

	Predicted 0	Predicted 1
True 0	5100	2942
True 1	681	1277

0.692

ROC-AUC

PR-AUC	0.345
Accuracy	0.638
Precision	0.303
Recall	0.652
F1-score	0.413

Best ROC-AUC and PR-AUC

Support Vector Classifier

MODEL RESULTS (TEST SET, 10,000 LOANS)

	Predicted 0	Predicted 1
True 0	5002	3040
True 1	670	1288

0.685

ROC-AUC

PR-AUC	0.326
Accuracy	0.629
Precision	0.298
Recall	0.658
F1-score	0.410

Highest recall

XGBoost

MODEL RESULTS (TEST SET, 10,000 LOANS)

	Predicted 0	Predicted 1
True 0	7780	262
True 1	1784	174

0.676

ROC-AUC

PR-AUC	0.321
Accuracy	0.795
Precision	0.399
Recall	0.089
F1-score	0.145

Highest accuracy, but misses 91% of defaults

Random Forest

MODEL RESULTS (TEST SET, 10,000 LOANS)

	Predicted 0	Predicted 1
True 0	7548	494
True 1	1676	282

0.672

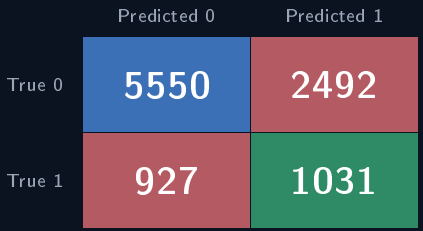
ROC-AUC

PR-AUC	0.309
Accuracy	0.783
Precision	0.363
Recall	0.144
F1-score	0.206

High accuracy, low recall

Neural Network

MODEL RESULTS (TEST SET, 10,000 LOANS)



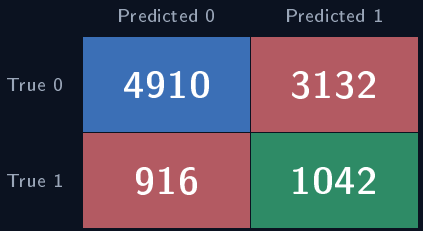
0.660

ROC-AUC

PR-AUC	0.311
Accuracy	0.658
Precision	0.293
Recall	0.527
F1-score	0.376

K-Nearest Neighbours

MODEL RESULTS (TEST SET, 10,000 LOANS)



0.603

ROC-AUC

PR-AUC	0.248
Accuracy	0.595
Precision	0.250
Recall	0.532
F1-score	0.340

Decision Tree

MODEL RESULTS (TEST SET, 10,000 LOANS)

	Predicted 0	Predicted 1
True 0	6212	1830
True 1	1363	595

0.538

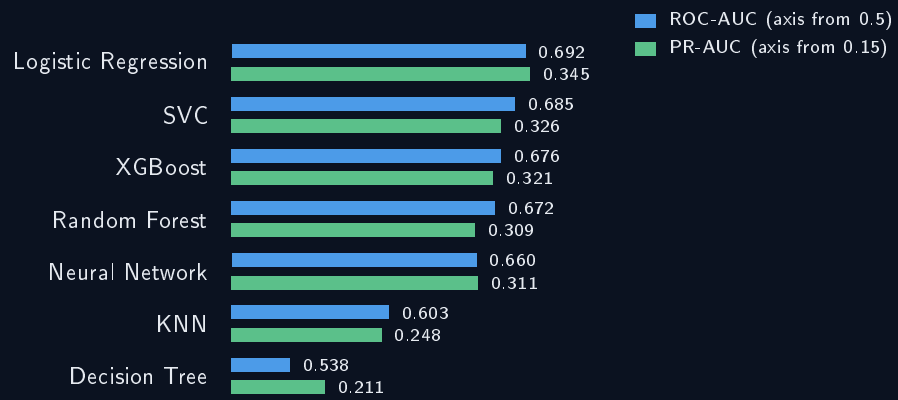
ROC-AUC

PR-AUC	0.211
Accuracy	0.681
Precision	0.245
Recall	0.304
F1-score	0.272

Close to a random classifier

Model performance

ROC-AUC AND PR-AUC COMPARISON



Model comparison

SUMMARY

Model	ROC-AUC	PR-AUC	Accuracy	Precision	Recall	F1
Logistic Regression	**0.692**	**0.345**	0.638	0.303	0.652	**0.413**
SVC	0.685	0.326	0.629	0.298	**0.658**	0.410
XGBoost	0.676	0.321	**0.795**	**0.399**	0.089	0.145
Random Forest	0.672	0.309	0.783	0.363	0.144	0.206
Neural Network	0.660	0.311	0.658	0.293	0.527	0.376
KNN	0.603	0.248	0.595	0.250	0.532	0.340
Decision Tree	0.538	0.211	0.681	0.245	0.304	0.272

Logistic regression gives the best ranking of risky loans (ROC-AUC 0.69) and is about 75% better than a random classifier at finding defaults (PR-AUC 0.35 vs. 0.20). Tree-based models reach the highest accuracy only by rarely predicting a default.

RGE — Logistic Regression

SAFE AI: EXPLAINABILITY

Highest-ranked features

1. Loan amount
2. Installment
3. Interest rate
4. DTI
5. Annual income
6. Home ownership: rent
7. Total accounts
8. Grade
9. Open accounts
10. Credit age

RGE compares the predictions of the full model with those of a model that excludes one feature at a time: the larger the change, the more the feature explains the model's output.

Loan size, repayment burden and price of credit drive the predictions.

RGR — Logistic Regression

SAFE AI: ROBUSTNESS

Feature	RGR	Feature	RGR
public_record_score	1.000	home_ownership_RENT	0.997
home_ownership_ANY	1.000	credit_age	0.995
pub_rec	1.000	open_acc	0.995
home_ownership_MORTGAGE	1.000	total_acc	0.993
loan_to_income_ratio	1.000	grade	0.993
addr_state	1.000	dti	0.983
has_record	1.000	annual_inc	0.983
delinq_2yrs	1.000	installment	0.925
inq_last_6mths	1.000	int_rate	0.924
fico_range_high / fico_avg	0.997	loan_amnt	0.887

Predictions are stable under perturbation for most variables (RGR close to 1). The model is more sensitive to its most important inputs: loan amount, interest rate and installment.

Limitations and next steps

CONCLUSIONS

Hyperparameters
Models were compared with default settings; tuning could improve tree-based models in particular.

Decision threshold
Fixed at 0.5. In practice it should reflect the cost of a missed default versus a rejected good borrower.

Scope
A modelling exercise on a sample of Lending Club data, not a production credit scoring system.

MACHINE LEARNING PROJECT

Thank you

for your attention

Federico Crivellari | Alan Sampaolo